

JAVA의 4가지 특징

- ✧ 캡슐화
- ✧ 상속
- ✧ 다형성
- ✧ 추상화

1. 상속

✧ 특징

- 부모 클래스의 기능을 자식 클래스가 물려받아 코드를 재사용하고 확장할 수 있는 개념
- 코드의 재사용성을 높여줌
- 상속을 통해 클래스 간의 계층적 구조 설계 가능

```
class 부모클래스 {  
    //부모클래스의 기능  
}
```

```
class 자식클래스 extends 부모클래스 {  
    //부모클래스의 기능 + 자식클래스의 기능  
}
```

✧ super와 this 키워드

- super 키워드

자식 클래스에서 부모 클래스의 멤버(필드,메소드)를 참조할 때 사용

부모 클래스의 생성자를 호출할 때에도 사용

```
class Animal{  
    String name;  
  
    Animal(String name){  
        This.name=name;  
    }  
  
    Void eat(){  
        System.out.println("먹이를 먹습니다.");  
    }  
}
```

```

class Dog extends Animal{
    String breed;

    Dog(String name, String breed){
        Super(name); //부모 생성자 호출
        This.breed = breed;
    }

    Void eat(){
        Super.eat(); //부모 메소드 호출
        System.out.println("강아지가 사료를 먹습니다.");
    }
}

```

■ this 키워드

자기 자신의 멤버(필드, 메소드)를 참조할 때 사용

자기 자신의 다른 생성자를 호출할 때 사용

✧ 메소드 오버라이딩

■ 메소드 오버로딩

같은 클래스 내에서 같은 이름의 메소드를 여러 개 정의하는 것

매개변수의 타입, 개수, 순서를 다르게 하여 구분할 수 있음

■ 메소드 오버라이딩

부모 클래스의 메소드를 자식 클래스에서 재정의하는 것

부모의 메소드를 자식 클래스에 맞게 수정해서 사용함

- 문제 : 상속을 사용해 일반 회원과 VIP 회원 만들기

```
package javaStudy;

class Member {
    String memberName; // 회원 이름
    String memberId; // 회원 아이디
    int point; // 포인트 점수

    // 생성자
    public Member(String memberName, String memberId) {
        this.memberName = memberName; // 회원 이름
        this.memberId = memberId; // 회원 아이디
        point = 0; // 최초 포인트는 0 점
    }

    //포인트 적립(구매금액의 1%)
    public void calcPrice(int price) {
        point += price*0.01;
    }

    //회원 정보 출력
    public void showMemberInfo() {
        System.out.println("==" + memberName + "님의 정보==");
        System.out.println("아이디 : " + memberId);
        System.out.println("포인트 : " + point + "점");
    }
}

class VIPMember extends Member{
    double saleRatio; //할인율
    String counselor; //전문 상담원

    //생성자
    public VIPMember(String memberName, String memberId, String
counselor) {
        super(memberName, memberId); //부모 클래스의 생성자 호출
        saleRatio = 0.1; //10% 할인
        this.counselor = counselor; //전문 상담원
    }

    //포인트 적립 오버라이딩(구매금액의 5%)
    @Override
    public void calcPrice(int price) {
        point += price*0.05;
    }
}
```

```

//VIP 회원 정보 출력(오버라이딩)
@Override
public void showMemberInfo() {
    super.showMemberInfo();
    System.out.println("할인율 : " + (int)(saleRatio*100) + "%");
    System.out.println("전문 상담원 : " + counselor);
}

}

public class Inheritance {
    public static void main(String[] args) {
        //일반 회원 생성
        Member member = new Member("홍길동", "hong");
        member.calcPrice(10000);
        member.showMemberInfo();

        System.out.println("\n");

        //VIP 회원 생성
        VIPMember vip = new VIPMember("김철수", "kim", "박상담");
        vip.calcPrice(10000);
        vip.showMemberInfo();
    }
}

```

출력값

```

==홍길동님의 정보==
아이디 : hong
포인트 : 100 점

==김철수님의 정보==
아이디 : kim
포인트 : 500 점
할인율 : 10%
전문 상담원 : 박상담

```

2. 다형성

✧ 특징

하나의 객체가 여러가지 타입을 가질 수 있는 것

다형성을 활용하면 부모 타입 참조변수로 자식 객체를 참조할 수 있음

✧ 다형성의 장점

- 코드의 유연성 증가
- 재사용성 향상
- 유지보수 용이

✧ 업캐스팅과 다운캐스팅

■ 업캐스팅

기본 자료형의 업캐스팅은 작은 자료형을 큰 자료형으로 변환하는 것

객체의 업캐스팅은 자식 객체를 부모 타입으로 변환하는 것

업캐스팅은 자동으로 변환됨! (묵시적 형변환)

```
Animal animal = new Dog(); //자동으로 업캐스팅
```

■ 다운캐스팅

기본 자료형의 다운캐스팅은 큰 자료형을 작은 자료형으로 변환하는 것

객체의 다운캐스팅은 부모 타입을 자식 타입으로 변환하는 것

명시적 형변환 필요

```
Animal animal = new Dog();
```

```
Dog dog = (Dog) animal; //명시적으로 다운캐스팅
```

■ instanceof

객체의 타입을 확인하는 연산자

다형성을 활용한 코드에서 사용할 수 있음

- 문제 : 다양한 결제수단을 통한 결제 프로세스 구현

```
package javaStudy;

class Payment {
    void processPayment(int amount) {
        System.out.println("결제 처리: " + amount + "원");
    }

    void refund(int amount) {
        System.out.println("환불 처리: " + amount + "원");
    }
}

class TossPayment extends Payment {
    void processPayment(int amount) {
        System.out.println("토스 결제 처리: " + amount + "원");
    }

    void refund(int amount) {
        System.out.println("토스 환불 처리: " + amount + "원");
    }
}

class KakaoPayment extends Payment {
    void processPayment(int amount) {
        System.out.println("카카오페이 결제 처리: " + amount + "원");
    }

    void refund(int amount) {
        System.out.println("카카오페이 환불 처리: " + amount + "원");
    }
}

class PaymentProcessor {

    void processPayment(Payment payment, int amount) {
        System.out.println("=== 결제를 시작합니다 ===");
        payment.processPayment(amount);
        System.out.println("=== 결제가 완료되었습니다 ===");
    }
}

public class PaymentExample {
    public static void main(String[] args) {
        // 결제 프로세서 생성
        PaymentProcessor processor = new PaymentProcessor();
    }
}
```

```

        // 여러 가지 결제 방식 생성
        Payment tossPayment = new TossPayment();
        Payment kakaoPayment = new KakaoPayment();

        // 결제 처리
        processor.processPayment(tossPayment, 10000);
        System.out.println();
        processor.processPayment(kakaoPayment, 50000);
    }
}

```

출력값

```

=== 결제를 시작합니다 ===
토스 결제 처리: 10000 원
=== 결제가 완료되었습니다 ===

=== 결제를 시작합니다 ===
카카오페이 결제 처리: 50000 원
=== 결제가 완료되었습니다 ===

```


3. 추상 클래스

✧ 특징

추상은 객체의 공통 특징이나 속성을 추출하는 개념

추상화하는 과정은 복잡한 것을 단순화하는 것이라고 볼 수 있음

내부 원리를 몰라도 조작 가능

✧ 추상클래스 구조

```
Abstract class 클래스명{  
  
    //필드  
  
    //생성자  
  
    //일반 메소드  
  
    //추상 메소드  
  
    Abstract void 메소드명();  
  
}
```

✧ 주의사항

New 연산자로 객체를 생성할 수 없음

자식 클래스에서 추상 클래스의 기능을 필수로 구현해야 함

✧ 추상 메소드

추상 클래스를 상속받은 자식 클래스가 필수로 구현해야 하는 메소드

메소드 바디가 없음!

```
abstract void method();
```

*공통 동작은 추상 클래스에서 직접 구현, 자식 클래스마다 세부 구현이 필요할 시 추상 메소드로 선언

4. 인터페이스

✧ 특징

객체의 동작을 표준화하는 기법

추상 메소드와 상수만 가질 수 있음

“어떤 동작을 해야 하는지”를 정의하고 구현 클래스에서 세부 로직을 구현

✧ 사용하는 이유

표준화 : 개발에 필요한 기본 틀 제공, 개발자들 간의 일관성 있는 개발 가능

다형성 구현 : 하나의 인터페이스로 다양한 구현이 가능, 객체의 교체가 쉬움

개발 독립성 : 인터페이스만 정의되면 개발 시작 가능, 다른 팀/개발자와 동시 개발 가능

유연성 : 기존 코드 변경 없이 새로운 기능 추가 가능, 독립적인 기능 구현 가능

✧ 추상 클래스와의 차이점

추상 클래스 : 일반 메소드, 추상 메소드 / 단일 상속만 가능 / 모든 종류의 필드 가능

인터페이스 : 추상 메소드, default 메소드 / 다중 구현 가능 / 상수만 가능

- 문제 : 인터페이스를 통한 다양한 할인 정책 구현

```
package javaStudy;

//할인 정책 인터페이스
interface DiscountPolicy {
    // 할인 금액 계산 메소드
    int calculateDiscount(int price);
}

//정률 할인 정책 구현(10% 할인)
class PercentDiscountPolicy implements DiscountPolicy {
    @Override
    public int calculateDiscount(int price) {
        return (int) (price * 0.1); // 10% 할인
    }
}

//정액 할인 정책 구현(1000 원 할인)
class FixedDiscountPolicy implements DiscountPolicy {
    @Override
    public int calculateDiscount(int price) {
        return 1000; // 1000 원 할인
    }
}

//할인 적용을 위한 상품 클래스
class Product {
    private String name;
    private int price;
    private DiscountPolicy discountPolicy;

    // 다형성을 사용해서 DiscountPolicy 인터페이스를 구현한 클래스를 전달
    받음
    public Product(String name, int price, DiscountPolicy
discountPolicy) {
        this.name = name;
        this.price = price;
        this.discountPolicy = discountPolicy;
    }

    // 할인된 가격 계산
    public int getDiscountPrice() {
        return price - discountPolicy.calculateDiscount(price);
    }

    // 상품 정보 출력
    public void printInfo() {
        System.out.println("상품명: " + name);
        System.out.println("원가: " + price + "원");
    }
}
```

```

        System.out.println("할인 금액: " +
discountPolicy.calculateDiscount(price) + "원");
        System.out.println("할인된 가격: " + getDiscountPrice() +
"원");
    }
}

//상품 할인 정책 구현 예제
public class DiscountExample {

    public static void main(String[] args) {
        // 할인 정책 객체 생성
        DiscountPolicy percentDiscount = new PercentDiscountPolicy();
        DiscountPolicy fixedDiscount = new FixedDiscountPolicy();

        // 상품 생성
        Product product1 = new Product("노트북", 20000,
percentDiscount);
        Product product2 = new Product("마우스", 5000,
fixedDiscount);

        // 상품 정보 출력
        System.out.println("===정률 할인 적용===");
        product1.printInfo();
        System.out.println();
        System.out.println("===정액 할인 적용===");
        product2.printInfo();

    }

}

```

출력값

```

===정률 할인 적용===
상품명: 노트북
원가: 20000 원
할인 금액: 2000 원
할인된 가격: 18000 원

===정액 할인 적용===
상품명: 마우스
원가: 5000 원
할인 금액: 1000 원
할인된 가격: 4000 원

```